
A Closer Look at Invalid Action Masking in Policy Gradient Algorithms

Shengyi Huang

College of Computing & Informatics
Drexel University
Philadelphia, PA 19104
sh3397@drexel.edu

Santiago Ontañón *

College of Computing & Informatics
Drexel University
Philadelphia, PA 19104
so367@drexel.edu

Abstract

In recent years, Deep Reinforcement Learning (DRL) algorithms have achieved state-of-the-art performance in many challenging strategy games. Because these games have complicated rules, an action sampled from the full discrete action space will typically be invalid. The usual approach to deal with this problem in policy gradient algorithms is to “mask out” invalid actions and just sample from the set of valid actions. The implications of this process, however, remain under-investigated. In this paper, we show that the standard working mechanism of invalid action masking corresponds to valid policy gradient updates. More interestingly, it works by applying a *state-dependent differentiable function* during the calculation of action probability distribution. Additionally, we show its critical importance to the performance of policy gradient algorithms. Specifically, our experiments show that invalid action masking scales well when the space of invalid actions is large, while the common approach of giving negative rewards for invalid actions will fail. Finally, we provide further insights by evaluating different action masking regimes, such as removing masking after an agent has been trained using masking.

1 Introduction

Deep Reinforcement Learning (DRL) algorithms have yielded state-of-the-art game playing agents in challenging domains such as Real-time Strategy (RTS) games [21, 20] and Multiplayer Online Battle Arena (MOBA) games [1, 23]. Because these games have complicated rules, the discrete action spaces of different states usually have different sizes. That is, one state might have 5 valid actions and another state might have 7 valid actions. To formulate these games as a standard reinforcement learning problem with a singular action set, previous work combine these discrete action spaces to a *full discrete action space* that contains available actions of all states [21, 1, 23]. Although such full discrete action space makes it easier to apply DRL algorithms, one issue is that an action sampled from this full discrete action space could be invalid for some game states, and this action will have to be discarded. To make matters worse, some games have extremely large full discrete action spaces and an action sampled will typically be invalid. As an example, the full discrete action space of Dota 2 has 1,837,080 dimensions [1], and an action sampled might be to buy an item when there is not enough gold. To avoid repeatedly sampling invalid actions in full discrete action spaces, recent work applies policy gradient algorithm in conjunction with a technique known as invalid action masking, which “masks out” invalid actions and then just sample from those actions that are valid [21, 1, 23]. To the best of our knowledge, however, the theoretical foundations of invalid action masking have not been studied and its empirical effect is under-investigated.

*Currently at Google

仔细研究策略梯度算法中的无效操作屏蔽

黄圣义
计算机与信息学院 德雷克塞尔大
学 费城, PA 19104 sh3397@drexel.
edu

Santiago Ontañón * 计算机
与信息学院 德雷克塞尔大学 费城,
PA 19104 so367@drexel.edu

抽象的

近年来, 深度强化学习 (DRL) 算法在许多具有挑战性的策略游戏中取得了最先进的性能。由于这些游戏具有复杂的规则, 因此从完整离散动作空间中采样的动作通常是无效的。在策略梯度算法中处理此问题的常用方法是“屏蔽”无效操作, 并仅从有效操作集中进行采样。然而, 这一过程的影响仍未得到充分研究。在本文中, 我们证明了无效动作屏蔽的标准工作机制对应于有效的策略梯度更新。更有趣的是, 它的工作原理是在计算动作概率分布时应用 *state-dependent differentiable function*。此外, 我们还展示了它对策略梯度算法的性能至关重要。具体来说, 我们的实验表明, 当无效动作的空间很大时, 无效动作屏蔽可以很好地扩展, 而对无效动作给予负面奖励的常见方法将会失败。最后, 我们通过评估不同的动作掩蔽机制来提供进一步的见解, 例如在使用掩蔽训练代理后去除掩蔽。

1 简介

深度强化学习 (DRL) 算法已经在具有挑战性的领域产生了最先进的游戏代理, 例如实时策略 (RTS) 游戏 [21, 20] 和多人在线战斗竞技场 (MOBA) 游戏 [1, 23]。由于这些游戏规则复杂, 不同状态的离散动作空间通常具有不同的大小。也就是说, 一个状态可能有 5 个有效动作, 而另一状态可能有 7 个有效动作。为了将这些游戏表述为具有单一动作集的标准强化学习问题, 之前的工作将这些离散动作空间组合成包含所有状态可用动作的 *full discrete action space* [21,1,23]。虽然这种完整的离散动作空间使得应用 DRL 算法变得更容易, 但一个问题是从这个完整的离散动作空间采样的动作对于某些游戏状态可能是无效的, 并且必须丢弃该动作。更糟糕的是, 一些游戏具有非常大的完整离散动作空间, 并且采样的动作通常是无效的。例如, Dota 2 的完整离散动作空间有 1,837,080 个维度 [1], 采样的动作可能是在没有足够金币时购买物品。为了避免在完全离散动作空间中重复采样无效动作, 最近的工作将策略梯度算法与称为无效动作屏蔽的技术结合使用, 该技术“屏蔽”无效动作, 然后仅从那些有效动作中进行采样[21,1,23]。然而, 据我们所知, 无效行为掩蔽的理论基础尚未得到研究, 其实证效果也尚未得到充分研究。

*Currently at Google

In this paper, we take a closer look at invalid action masking, pointing out the gradient produced by invalid action masking corresponds to a valid policy gradient. More interestingly, we show that in fact, invalid action masking can be seen as applying a *state-dependent differentiable function* during the calculation of action probability distribution, to produce a behavior policy. Next, we design experiments to compare the performance of *invalid action masking* versus *invalid action penalty*, which is a common approach that gives negative rewards for invalid actions so that the agent learns to maximize reward by not executing any invalid actions. We empirically show that, when the space of invalid actions grows, invalid action masking scales well and the agent solves our desired task while invalid action penalty struggles to explore even the very first reward. Then, we design experiments to answer two questions: (1) What happens if we remove the invalid action mask once the agent was trained with the mask? (2) What is the agent’s performance when we implement the invalid action masking naively by sampling the action from the masked action probability distribution but updating the policy gradient using the unmasked action probability distribution? Finally, we made our source code available at GitHub for the purpose of reproducibility².

2 Background

We consider the Reinforcement Learning problem in a Markov Decision Process (MDP) denoted as $(S, A, P, \rho_0, r, \gamma, T)$, where S is the state space, A is the discrete action space, $P : S \times A \times S \rightarrow [0, 1]$ is the state transition probability, $\rho_0 : S \rightarrow [0, 1]$ is the initial state distribution, $r : S \times A \rightarrow \mathbb{R}$ is the reward function, γ is the discount factor, and T is the maximum episode length. A stochastic policy $\pi_\theta : S \times A \rightarrow [0, 1]$, parameterized by a parameter vector θ , assigns a probability value to an action given a state. The goal is to maximize the expected discounted return:

$$J = \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \text{ where } \tau \text{ is the trajectory } (s_0, a_0, r_0, s_1, \dots, s_{T-1}, a_{T-1}, r_{T-1})$$

and $s_0 \sim \rho_0, s_t \sim P(\cdot | s_{t-1}, a_{t-1}), a_t \sim \pi_\theta(\cdot | s_t), r_t = r(s_t, a_t)$

The notation $P(\cdot | s_{t-1}, a_{t-1})$ represents the states transition distribution given the previous state s_{t-1} and action a_{t-1} , and the notation $s_t \sim P(\cdot | s_{t-1}, a_{t-1})$ represents that the state s_t visited at time t is sampled from $P(\cdot | s_{t-1}, a_{t-1})$. Similarly, $\pi_\theta(\cdot | s_t)$ represents the action distribution given state s_t , and $a_t \sim \pi_\theta(\cdot | s_t)$ means the action a_t at time t is sampled from $\pi_\theta(\cdot | s_t)$.

Policy Gradient Algorithms. The core idea behind policy gradient algorithms is to obtain the *policy gradient* $\nabla_\theta J$ of the expected discounted return with respect to the policy parameter θ . Doing gradient ascent $\theta = \theta + \nabla_\theta J$ therefore maximizes the expected discounted reward. Earlier work proposed the following policy gradient estimate to the objective J [19, 18]:

$$g_{\text{policy}} = \mathbb{E}_\tau [\nabla_\theta \log \pi_\theta(a_\tau | s_\tau) G_\tau] = \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) G_t \right], \quad (1)$$

where $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ denotes the discounted return following time t .

3 Invalid Action Masking

Invalid action masking is a common technique implemented to avoid repeatedly generating invalid actions in large discrete action spaces [21, 1, 23]. To the best of our knowledge, there is no literature providing detailed descriptions of the implementation of invalid action masking. Existing work [21, 1] seems to treat invalid action masking as an auxiliary detail, usually describing it using only a few sentences. Additionally, there is no literature providing theoretical justification to explain why it works with policy gradient algorithms. In this section, we examine how invalid action masking is implemented and prove it indeed corresponds to valid policy gradient updates [19]. More interestingly, we show it works by applying a *state-dependent differentiable function* during the calculation of action probability distribution.

First, let us see how a discrete action is typically generated through policy gradient algorithms. Most policy gradient algorithms employ a neural network to represent the policy, which usually

²<https://github.com/vwxyzjn/invalid-action-masking>

在本文中，我们仔细研究了无效动作屏蔽，指出无效动作屏蔽产生的梯度对应于有效的策略梯度。更有趣的是，我们表明，事实上，无效动作屏蔽可以被视为在计算动作概率分布期间应用 *state-dependent differentiable function*，以产生行为策略。接下来，我们设计实验来比较 *invalid action masking* 与 *invalid action penalty* 的性能，这是一种常见的方法，会对无效动作给予负面奖励，以便代理学习通过不执行任何无效动作来最大化奖励。我们凭经验表明，当无效动作的空间增加时，无效动作掩蔽会很好地扩展，并且代理可以解决我们期望的任务，而无效动作惩罚甚至很难探索第一个奖励。然后，我们设计实验来回答两个问题：（1）一旦使用掩模训练智能体，如果我们删除无效的动作掩模，会发生什么？（2）当我们通过从屏蔽动作概率分布中采样动作来简单地实现无效动作屏蔽，但使用未屏蔽动作概率分布更新策略梯度时，代理的性能如何？最后，出于可重复性的目的，我们在 GitHub 上提供了源代码²。

2 背景

我们考虑马尔可夫决策过程（MDP）中的强化学习问题，表示为 $(S, A, P, \rho_0, r, \gamma, T)$ ，其中 S 是状态空间， A 是离散动作空间， $P: S \times A \times S \rightarrow [0,1]$ 是状态转移概率， $\rho_0: S \rightarrow [0,1]$ 是初始状态分布， $r: S \times A \rightarrow \mathbb{R}$ 是奖励函数， γ 是折扣因子， T 是最大剧集长度。随机策略 $\pi_\theta: S \times A \rightarrow [0, 1]$ 由参数向量 θ 参数化，为给定状态的动作分配概率值。目标是最大化预期折扣回报：

$$J = \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \text{ where } \tau \text{ is the trajectory } (s_0, a_0, r_0, s_1, \dots, s_{T-1}, a_{T-1}, r_{T-1})$$

and $s_0 \sim \rho_0, s_t \sim P(\cdot | s_{t-1}, a_{t-1}), a_t \sim \pi_\theta(\cdot | s_t), r_t = r(s_t, a_t)$

符号 $P(\cdot | s_{t-1}, a_{t-1})$ 表示给定先前状态 s_{t-1} 和动作 a_{t-1} 的状态转换分布，符号 $s_t \sim P(\cdot | s_{t-1}, a_{t-1})$ 表示从 $P(\cdot | s_{t-1}, a_{t-1})$ 采样在时间 t 访问的状态 s_t 。类似地， $\pi_\theta(\cdot | s_t)$ 表示给定状态 s_t 的动作分布，而 $a_t \sim \pi_\theta(\cdot | s_t)$ 表示从 $\pi_\theta(\cdot | s_t)$ 采样在时间 t 的动作 a_t 。

策略梯度算法。策略梯度算法的核心思想是获取预期贴现收益相对于策略参数 θ 的策略梯度 $\nabla_\theta J$ 。因此，进行梯度上升 $\theta = \theta + \nabla_\theta J$ 可以最大化预期的折扣奖励。早期的工作提出了对目标 J [19, 18] 的以下策略梯度估计：

$$g_{\text{policy}} = \mathbb{E}_\tau [\nabla_\theta \log \pi_\theta(a_\tau | s_\tau) G_\tau] = \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) G_t \right], \quad (1)$$

其中 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ 表示时间 t 后的折扣退货。

3 无效动作屏蔽

无效动作屏蔽是一种常用技术，用于避免在大型离散动作空间中重复生成无效动作[21,1,23]。据我们所知，没有文献详细描述无效动作屏蔽的实现。现有的工作 [21, 1] 似乎将无效动作屏蔽视为辅助细节，通常仅使用几句话来描述它。此外，没有文献提供理论依据来解释为什么它与策略梯度算法一起工作。在本节中，我们研究如何实现无效动作屏蔽，并证明它确实对应于有效的策略梯度更新[19]。更有趣的是，我们通过在计算动作概率分布期间应用 *state-dependent differentiable function* 来展示它的工作原理。

首先，让我们看看离散动作通常是如何通过策略梯度算法生成的。大多数策略梯度算法采用神经网络来表示策略，通常

²<https://github.com/vwxyzjn/invalid-action-masking>

outputs unnormalized scores (logits) and then converts them into an action probability distribution using a softmax operation or equivalent, which is the framework we will assume in the rest of the paper. For illustration purposes, consider an MDP with the action set $A = \{a_0, a_1, a_2, a_3\}$ and $S = \{s_0, s_1\}$, where the MDP reaches the terminal state s_1 immediately after an action is taken in the initial state s_0 and the reward is always +1. Further, consider a policy π_θ parameterized by $\theta = [l_0, l_1, l_2, l_3] = [1.0, 1.0, 1.0, 1.0]$ that, for the sake of this example, directly produces θ as the output logits. Then in s_0 we have:

$$\begin{aligned}\pi_\theta(\cdot|s_0) &= [\pi_\theta(a_0|s_0), \pi_\theta(a_1|s_0), \pi_\theta(a_2|s_0), \pi_\theta(a_3|s_0)] = \text{softmax}([l_0, l_1, l_2, l_3]) \\ &= [0.25, 0.25, 0.25, 0.25], \quad \pi_\theta(a_i|s_0) = \frac{\exp(l_i)}{\sum_j \exp(l_j)}\end{aligned}\quad (2)$$

At this point, regular policy gradient algorithms will sample an action from $\pi_\theta(\cdot|s_0)$. Suppose a_0 is sampled from $\pi_\theta(\cdot|s_0)$, and the policy gradient can be calculated as follows:

$$\begin{aligned}g_{\text{policy}} &= \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) G_t \right] \\ &= \nabla_\theta \log \pi_\theta(a_0|s_0) G_0 \\ &= [0.75, -0.25, -0.25, -0.25]\end{aligned}\quad (\nabla_\theta \log \text{softmax}(\theta)_j)_i = \begin{cases} (1 - \frac{\exp(l_j)}{\sum_j \exp(l_j)}) & \text{if } i = j \\ \frac{-\exp(l_j)}{\sum_j \exp(l_j)} & \text{otherwise} \end{cases}$$

Now suppose a_2 is invalid for state s_0 , and the only valid actions are a_0, a_1, a_3 . Invalid action masking helps to avoid sampling invalid actions by “masking out” the logits corresponding to the invalid actions. This is usually accomplished by replacing the logits of the actions to be masked by a large negative number M (e.g. $M = -1 \times 10^8$). Let us use inv_s to denote this masking process and we can calculate the re-normalized probability distribution $\pi'_\theta(\cdot|s_0)$ as the following:

$$\pi'_\theta(\cdot|s_0) = \text{softmax}(\text{inv}_s([l_0, l_1, l_2, l_3])) \quad (3)$$

$$\begin{aligned}&= \text{softmax}([l_0, l_1, M, l_3]) = [\pi'_\theta(a_0|s_0), \pi'_\theta(a_1|s_0), \epsilon, \pi'_\theta(a_3|s_0)] \\ &= [0.33, 0.33, 0.0000, 0.33]\end{aligned}\quad (4)$$

where ϵ is the resulting probability of the masked invalid action, which should be a small number. If M is chosen to be sufficiently negative, the probability of choosing the masked invalid action a_2 will be virtually zero. After finishing the episode, the policy is updated according to the following gradient, which we refer to as the *invalid action policy gradient*.

$$\begin{aligned}g_{\text{invalid action policy}} &= \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi'_\theta(a_t|s_t) G_t \right] \\ &= \nabla_\theta \log \pi'_\theta(a_0|s_0) G_0 = [0.67, -0.33, 0.0000, -0.33]\end{aligned}\quad (5)$$

This example highlights that invalid action masking appears to do more than just “renormalizing the probability distribution”; it in fact makes the gradient corresponding to the logits of the invalid action to zero.

3.1 Invalid Action Masking Produces a Valid Policy Gradient

The action selection process is affected by a process that seems external to π_θ that calculates the mask. It is therefore natural to wonder how does the policy gradient theorem [19] apply. As a matter of fact, our analysis shows that the process of invalid action masking can be considered as a state-dependent differentiable function applied for the calculation of π'_θ , and therefore $g_{\text{invalid action policy}}$ can be considered as a policy gradient update for π'_θ .

Proposition 1. *$g_{\text{invalid action policy}}$ is the policy gradient of policy π'_θ .*

Proof. Let $s \in S$ to be arbitrary and consider the process of invalid action masking as a differentiable function inv_s to be applied to the logits $l(s)$ outputted by policy π_θ given state s . Then we have:

$$\begin{aligned}\pi'_\theta(\cdot|s_t) &= \text{softmax}(\text{inv}_s(l(s))) \\ \text{inv}_s(l(s))_i &= \begin{cases} l_i & \text{if } a_i \text{ is valid in } s \\ M & \text{otherwise} \end{cases}\end{aligned}$$

输出非标准化分数 (logits)，然后使用 softmax 操作或等效操作将它们转换为动作概率分布，这是我们将在本文其余部分假设的框架。出于说明目的，考虑具有操作集 $A = \{a_0, a_1, a_2, a_3\}$ 和 $S = \{s_0, s_1\}$ 的 MDP，其中在初始状态 s_0 中采取操作后，MDP 立即到达最终状态 s_1 ，并且奖励始终为 +1。此外，考虑由 $\theta = [l_0, l_1, l_2, l_3] = [1.0, 1.0, 1.0, 1.0]$ 参数化的策略 π_θ ，在本示例中，直接生成 θ 作为输出 logits。然后在 s_0 中我们有：

$$\begin{aligned}\pi_\theta(\cdot|s_0) &= [\pi_\theta(a_0|s_0), \pi_\theta(a_1|s_0), \pi_\theta(a_2|s_0), \pi_\theta(a_3|s_0)] = \text{softmax}([l_0, l_1, l_2, l_3]) \\ &= [0.25, 0.25, 0.25, 0.25], \quad \pi_\theta(a_i|s_0) = \frac{\exp(l_i)}{\sum_j \exp(l_j)}\end{aligned}\quad (2)$$

此时，常规策略梯度算法将从 $\pi_\theta(\cdot|s_0)$ 中采样动作。假设 a_0 是从 $\pi_\theta(\cdot|s_0)$ 中采样的，策略梯度可以计算如下：

$$\begin{aligned}g_{\text{policy}} &= \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) G_t \right] \\ &= \nabla_\theta \log \pi_\theta(a_0|s_0) G_0 \\ &= [0.75, -0.25, -0.25, -0.25]\end{aligned}\quad (\nabla_\theta \log \text{softmax}(\theta)_j)_i = \begin{cases} (1 - \frac{\exp(l_j)}{\sum_j \exp(l_j)}) & \text{if } i = j \\ \frac{-\exp(l_j)}{\sum_j \exp(l_j)} & \text{otherwise} \end{cases}$$

现在假设 a_2 对于状态 s_0 无效，唯一有效的操作是 a_0, a_1, a_3 。无效操作屏蔽通过“屏蔽”与无效操作相对应的 logits 来帮助避免对无效操作进行采样。这通常是通过用大负数 M (替换要屏蔽的操作的 logits 来实现的，例如 $M = -1 \times 10^8$)。让我们用 inv_s 来表示这个掩蔽过程，我们可以计算重归一化概率分布 $\pi'_\theta(\cdot|s_0)$ 如下：

$$\pi'_\theta(\cdot|s_0) = \text{softmax}(\text{inv}_s([l_0, l_1, l_2, l_3])) \quad (3)$$

$$\begin{aligned}&= \text{softmax}([l_0, l_1, M, l_3]) = [\pi'_\theta(a_0|s_0), \pi'_\theta(a_1|s_0), \epsilon, \pi'_\theta(a_3|s_0)] \\ &= [0.33, 0.33, 0.0000, 0.33]\end{aligned}\quad (4)$$

其中 ϵ 是屏蔽无效操作的结果概率，它应该是一个很小的数字。如果 M 选择为足够负，则选择屏蔽无效动作 a_2 的概率几乎为零。完成该 episode 后，策略根据以下梯度进行更新，我们将其称为 *invalid action policy gradient*。

$$\begin{aligned}g_{\text{invalid action policy}} &= \mathbb{E}_\tau \left[\nabla_\theta \sum_{t=0}^{T-1} \log \pi'_\theta(a_t|s_t) G_t \right] \\ &= \nabla_\theta \log \pi'_\theta(a_0|s_0) G_0 = [0.67, -0.33, 0.0000, -0.33]\end{aligned}\quad (5)$$

这个例子强调了无效动作屏蔽似乎不仅仅是“重新规范化概率分布”；它实际上使与无效动作的逻辑对应的梯度为零。

3.1 无效动作屏蔽产生有效的策略梯度

动作选择过程受到计算掩码的 π_θ 外部过程的影响。因此，很自然地想知道策略梯度定理 [19] 是如何应用的。事实上，我们的分析表明，无效动作屏蔽的过程可以被视为应用于计算 π'_θ 的状态相关可微函数，因此 $g_{\text{invalid action policy}}$ 可以被视为 π'_θ 的策略梯度更新。

命题 1. *$g_{\text{invalid action policy}}$ is the policy gradient of policy π'_θ .*

Proof. 令 $s \in S$ 是任意的，并将无效动作屏蔽的过程视为可微函数 inv_s ，应用于给定状态 s 的策略 π_θ 输出的 logits $l(s)$ 。然后我们有：

$$\begin{aligned}\pi'_\theta(\cdot|s_t) &= \text{softmax}(\text{inv}_s(l(s))) \\ \text{inv}_s(l(s))_i &= \begin{cases} l_i & \text{if } a_i \text{ is valid in } s \\ M & \text{otherwise} \end{cases}\end{aligned}$$

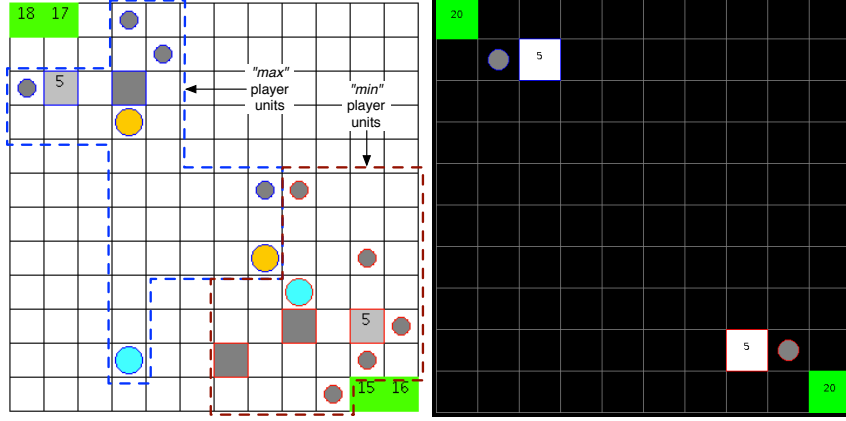


Figure 1: On the left is a screenshot of μ RTS . Square units are “bases” (light grey, that can produce workers), “barracks” (dark grey, that can produce military units), and “resources mines” (green, from where workers can extract resources to produce more units), the circular units are “workers” (small, dark grey) and military units (large, yellow or light blue), and on the right is the 10×10 map we used to train agents to harvest resources. The agents could control units at the top left, and the units in the bottom left will remain stationary.

Clearly, inv_{s_t} applies either an identity function or a constant function for elements in the logits. Since these two kinds of functions are differentiable, inv_s is differentiable. Therefore, π'_θ is differentiable to its parameters θ . That is, $\frac{\partial \pi'_\theta(a|s)}{\partial \theta}$ exists for all $a \in A, s \in S$, which satisfies the assumption of policy gradient theorem [19]. Hence, $g_{\text{invalid action policy}}$ is the policy gradient of policy π'_θ . $\square$

Although Proposition 1 suggests invalid action masking is theoretically supported by policy gradient theorem [19], note that inv_s is a state-dependent differentiable function. That is, given a vector x of length $|A|$ and two states s, s' with different number of invalid actions available in these states, $\text{inv}_s(x) \neq \text{inv}_{s'}(x)$.

4 Experimental Setup

In the remaining of this paper, we provide a series of empirical results showing the practical implications of invalid action masking.

4.1 Evaluation Environment

We use μ RTS³ as our testbed, which is a minimalistic RTS game maintaining the core features that make RTS games challenging from an AI point of view: simultaneous and durative actions, large branching factors and real-time decision making. A screenshot of the game can be found in Figure 1. It is the perfect testbed for our experiments because the action space in μ RTS grows combinatorially and so does the number of invalid actions that could be generated by the DRL agent. We now present the technical details of the environment for our experiments.

- **Observation Space.** Given a map of size $h \times w$, the observation is a tensor of shape (h, w, n_f) , where n_f is a number of feature planes that have binary values. The observation space used in this paper uses 27 feature planes as shown in Table 3 in the Appendices, similar to previous work in μ RTS [17, 22, 8]. A feature plane can be thought of as a concatenation of multiple one-hot encoded features. As an example, if there is a worker with hit points equal to 1, not carrying any resources, owner being Player 1, and currently not executing any actions, then the one-hot encoding features will look like the following:

$$[0, 1, 0, 0, 0], [1, 0, 0, 0, 0], [1, 0, 0], [0, 0, 0, 0, 1, 0, 0, 0], [1, 0, 0, 0, 0, 0, 0]$$

³<https://github.com/santiontanon/microrts>

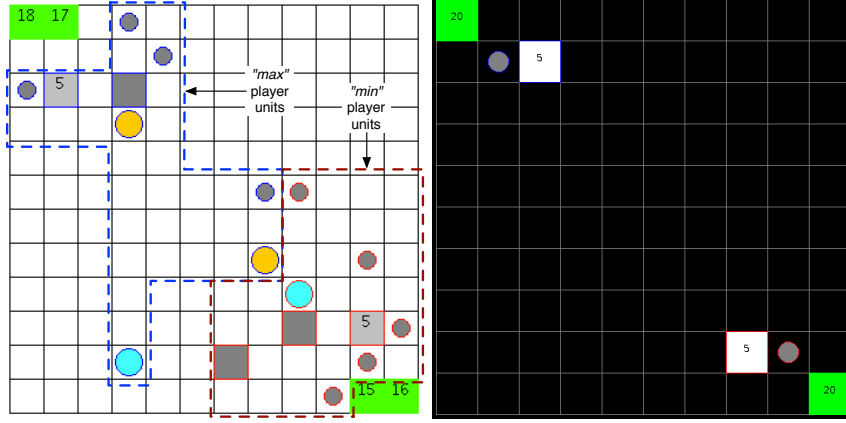


图 1: 左侧是 μ RTS 的屏幕截图。方形单位是“基地”（浅灰色，可以生产工人）、“军营”（深灰色，可以生产军事单位）和“资源矿”（绿色，工人可以从中提取资源来生产更多单位），圆形单位是“工人”（小，深灰色）和军事单位（大，黄色或浅蓝色），右边是我们用来训练智能体收获资源的 10×10 地图。代理可以控制左上角的单位，左下角的单位将保持静止。

显然， inv_{s_t} 对 logits 中的元素应用恒等函数或常量函数。由于这两种函数是可微的，因此 inv_s 是可微的。因此， π'_θ 与其参数 θ 可微。也就是说，所有 $a \in A, s \in S$ 都存在 $\frac{\partial \pi'_\theta(a|s)}{\partial \theta}$ ，满足策略梯度定理的假设[19]。因此， $g_{\text{invalid action policy}}$ 是策略 π'_θ 的策略梯度。 $\square$

尽管命题 1 表明策略梯度定理 [19] 理论上支持无效动作屏蔽，但请注意， inv_s 是一个与状态相关的可微函数。也就是说，给定长度为 $|A|$ 的向量 x 和两个状态 s, s' ，这些状态中可用的无效动作数量不同，即 $\text{inv}_s(x) \neq \text{inv}_{s'}(x)$ 。

4 实验装置

在本文的其余部分，我们提供了一系列实证结果，展示了无效动作屏蔽的实际含义。

4.1 评估环境

我们使用 μ RTS³ 作为测试平台，这是一款简约的 RTS 游戏，保留了从 AI 角度来看使 RTS 游戏具有挑战性的核心功能：同时和持续的动作、大的分支因素和实时决策。游戏的屏幕截图如图 1 所示。它是我们实验的完美测试平台，因为 μ RTS 中的动作空间会组合增长，DRL 代理可能生成的无效动作数量也会增长。我们现在介绍我们实验环境的技术细节。

- 观察空间。给定大小为 $h \times w$ 的地图，观察结果是形状为 (h, w, n_f) 的张量，其中 n_f 是具有二进制值的特征平面的数量。本文使用的观察空间使用 27 个特征平面，如附录中的表 3 所示，类似于 μ RTS [17,22,8] 中的先前工作。特征平面可以被认为是多个 one-hot 编码特征的串联。举个例子，如果有一个工人的生命值等于 1，没有携带任何资源，所有者是玩家 1，并且当前没有执行任何操作，那么 one-hot 编码特征将如下所示：

$$[0, 1, 0, 0, 0], [1, 0, 0, 0, 0], [1, 0, 0], [0, 0, 0, 0, 1, 0, 0, 0], [1, 0, 0, 0, 0, 0]$$

³<https://github.com/santiontanon/microrts>

Table 1: The action components and their descriptions.

Action Components	Range	Description
Source Unit	$[0, h \times w - 1]$	the location of unit selected to perform an action
Action Type	$[0, 5]$	NOOP, move, harvest, return, produce, attack
Move Parameter	$[0, 3]$	north, east, south, west
Harvest Parameter	$[0, 3]$	north, east, south, west
Return Parameter	$[0, 3]$	north, east, south, west
Produce Direction Parameter	$[0, 3]$	north, east, south, west
Produce Type Parameter	$[0, 5]$	resource, base, barrack, worker, light, heavy, ranged
Attack Target Unit	$[0, h \times w - 1]$	the location of unit that will be attacked

The 27 values of each feature plane for the position in the map of such worker will thus be:

$$[0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0]$$

- **Action Space.** Given a map of size $h \times w$, the action is an 8-dimensional vector of discrete values as specified in Table 1. The action space is designed similar to the action space formulation by Hausknecht, et al., [7]. The first component of the action vector represents the unit in the map to issue actions to, the second is the action type, and the rest of components represent the different parameters different action types can take. Depending on which action type is selected, the game engine will use the corresponding parameters to execute the action. Additional details on how to interface actions with μ RTS internally can be found at Appendix B.
- **Rewards.** We are evaluating our agents on the simple task of harvesting resources as fast as they can for Player 1 who controls units at the top left of the map. A +1 reward is given when a worker harvests a resource, and another +1 is received once the worker returns the resource to a base.
- **Termination Condition.** We set the maximum game length to be of 200 time steps, but the game could be terminated earlier if the all of the resources in the map are harvested first.

Notice that the space of invalid actions becomes significantly larger in larger maps. This is because the range of the first and last discrete values in the action space, corresponding to *Source Unit* and *Attack Target Unit* selection, grows linearly with the size of the map. To illustrate, in our experiments, there are usually only two units that can be selected as the *Source Unit* (the base and the worker). Although it is possible to produce more units or buildings to be selected, the production behavior has no contribution to reward and therefore is generally not learned by the agent. Note the range of *Source Unit* is $4 \times 4 = 16$ and $24 \times 24 = 576$, in maps of size 4×4 and 24×24 , respectively. Selecting a valid *Source Unit* at random has a probability of $2/16 = 0.125$ in the 4×4 map and $2/576 = 0.0034$ in the 24×24 map. With such action space, we can examine the scalability of invalid action masking.

4.2 Training Algorithm

We use Proximal Policy Optimization [16] as the DRL algorithm to train our agents. The details of the implementation and neural network architecture, hyperparameters can be found in Appendix A.

4.3 Strategies to Handle Invalid Actions

To examine the empirical importance of invalid action masking, we compare the following four strategies to handle invalid actions.

1. **Invalid action penalty.** Every time the agent issues an invalid action, the game environment adds a non-positive reward $r_{\text{invalid}} \leq 0$ to the reward produced by the current time step. This technique is standard in previous work [4]. We experiment with $r_{\text{invalid}} \in \{0, -0.01, -0.1, -1\}$, respectively, to study the effect of the different scales on the negative reward.
2. **Invalid action masking.** At each time step t , the agent receives a mask on the *Source Unit* and *Attack Target Unit* features such that only valid units can be selected and targeted. Note that in our experiments, invalid actions still could be sampled because the agent could still

表 1: 操作组件及其描述。

Action Components	Range	Description
Source Unit	$[0, h \times w - 1]$	the location of unit selected to perform an action
Action Type	$[0, 5]$	NOOP, move, harvest, return, produce, attack
Move Parameter	$[0, 3]$	north, east, south, west
Harvest Parameter	$[0, 3]$	north, east, south, west
Return Parameter	$[0, 3]$	north, east, south, west
Produce Direction Parameter	$[0, 3]$	north, east, south, west
Produce Type Parameter	$[0, 5]$	resource, base, barrack, worker, light, heavy, ranged
Attack Target Unit	$[0, h \times w - 1]$	the location of unit that will be attacked

因此，该工人在地图中的位置的每个特征平面的 27 个值将是：

$$[0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0]$$

- 行动空间。给定大小为 $h \times w$ 的映射，动作是离散值的 8 维向量，如表 1 所示。动作空间的设计类似于 Hausknecht 等人 [7] 的动作空间公式。动作向量的第一个分量表示映射中要向其发出动作的单元，第二个分量是动作类型，其余分量表示不同动作类型可以采用的不同参数。根据选择的动作类型，游戏引擎将使用相应的参数来执行动作。有关如何在内部将操作与 μ RTS 连接的更多详细信息，请参阅附录 B。
- 奖励。我们正在评估我们的代理的简单任务，即为控制地图左上方单位的玩家 1 尽快收获资源。当工人收获资源时，会获得 +1 奖励；一旦工人将资源返回基地，就会收到另一个 +1 奖励。
- 终止条件。我们将最大游戏长度设置为 200 个时间步长，但如果首先收获地图中的所有资源，游戏可以提前终止。

请注意，在较大的地图中，无效操作的空间明显变大。这是因为动作空间中第一个和最后一个离散值的范围（对应于 *Source Unit* 和 *Attack Target Unit* 选择）随着映射的大小线性增长。为了说明这一点，在我们的实验中，通常只有两个单元可以选择作为 *Source Unit*（基础和工作单元）。尽管可以生产更多的单位或建筑物以供选择，但生产行为对奖励没有贡献，因此通常不会被代理学习。请注意，在大小为 4×4 和 24×24 的地图中，*Source Unit* 的范围分别为 $4 \times 4 = 16$ 和 $24 \times 24 = 576$ 。随机选择有效的 *Source Unit* 在 4×4 映射中的概率为 $2/16 = 0.125$ ，在 24×24 映射中的概率为 $2/576 = 0.0034$ 。通过这样的动作空间，我们可以检查无效动作屏蔽的可扩展性。

4.2 训练算法

我们使用近端策略优化[16]作为 DRL 算法来训练我们的代理。实现和神经网络架构、超参数的详细信息可以在附录 A 中找到。

4.3 处理无效动作的策略

为了检验无效操作屏蔽的经验重要性，我们比较了以下四种处理无效操作的策略。

- 1、无效动作处罚。每次代理发出无效动作时，游戏环境都会在当前时间步产生的奖励中添加一个非正奖励 $r_{\text{invalid}} \leq 0$ 。该技术是以前工作中的标准技术[4]。我们分别用 $r_{\text{invalid}} \in \{0, -0.01, -0.1, -1\}$ 进行实验，以研究不同尺度对负奖励的影响。
- 2、无效的动作屏蔽。在每个时间步骤 t ，代理都会收到 *Source Unit* 和 *Attack Target Unit* 特征的掩码，以便只能选择和定位有效的单元。请注意，在我们的实验中，仍然可以对无效操作进行采样，因为代理仍然可以

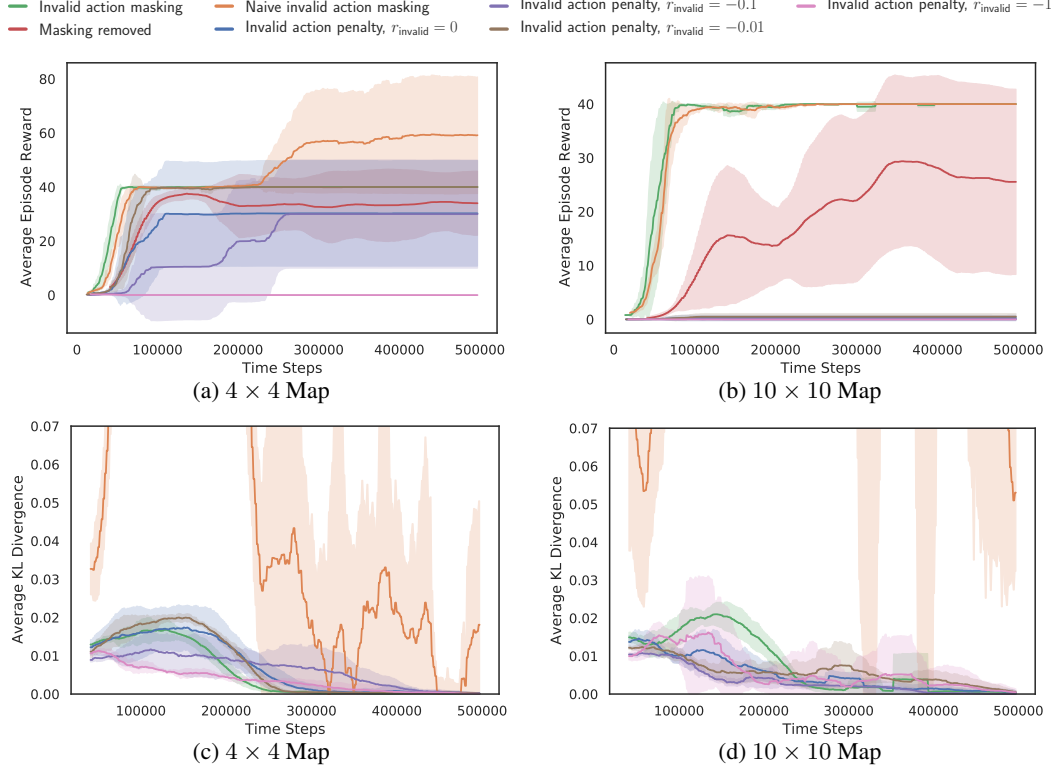


Figure 2: (a) and (b) show the learning curves of agents with different strategies to handle invalid actions. The x-axis shows the number of game steps and y-axis shows the average episode reward gathered. (c) and (d) have the x-axis showing the number of game steps and y-axis showing the average Kullback–Leibler (KL) divergence between the target and current policy of PPO. Shaded area represents one standard deviation of the data over 4 random seeds. Curves are smoothed for readability. For results in other maps, see Figure 3 in the Appendices.

select incorrect parameters for the current action type. We didn’t provide a feature-complete invalid action mask for simplicity, as the mask on *Source Unit* and *Attack Target Unit* already significantly reduce the action space.

3. **Naive invalid action masking.** At each time step t , the agent receives the same mask on the *Source Unit* and *Attack Target Unit* as described for invalid action masking. The action shall still be sampled according to the re-normalized probability calculated in Equation 4, which ensures no invalid actions could be sampled, but the gradient is updated according to probability calculated in Equation 2. We call this implementation *naive invalid action masking* because its gradient does not replace the gradient of the logits corresponds to invalid actions with zero.
4. **Masking removed.** At each time step t , the agent receives the same mask on the *Source Unit* and *Attack Target Unit* as described for invalid action masking, and trains in the same way as the agent trained under invalid action masking. However, we then evaluate the agent without providing the mask. In other words, in this scenario, we evaluate what happens if we train with a mask, but then perform without it.

We evaluate the agent’s performance in maps of sizes 4×4 , 10×10 , 16×16 , and 24×24 . All maps have one base and one worker for each player, and each worker is located near the resources.

4.4 Evaluation Metrics

We used the following metrics to measure the performance of the agents in our experiments: r_{episode} is the average episode reward over last 10 episodes. a_{null} is the average number of actions that select a *Source Unit* that is not valid over last 10 episodes. a_{busy} is the average number of actions that select

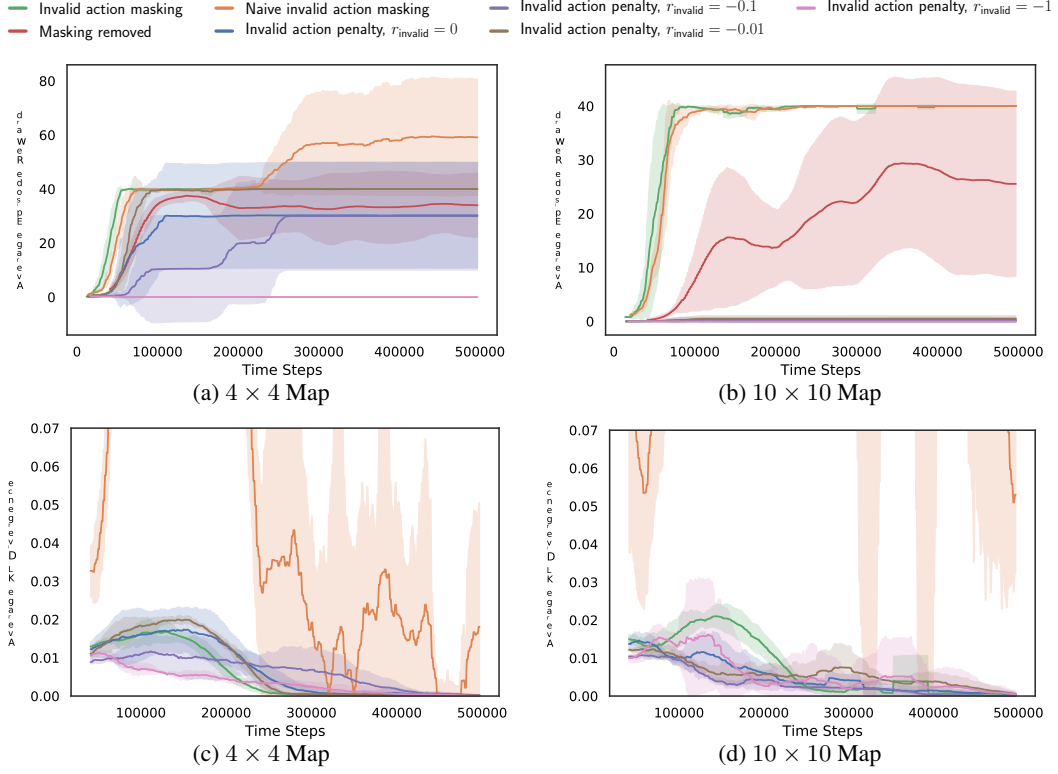


图 2: (a) 和 (b) 显示了采用不同策略处理无效动作的智能体的学习曲线。x 轴显示游戏步骤数, y 轴显示收集的平均剧集奖励。(c) 和 (d) 的 x 轴显示游戏步数, y 轴显示 PPO 目标和当前策略之间的平均 Kullback-Leibler (KL) 差异。阴影区域代表 4 个随机种子数据的一个标准差。曲线经过平滑处理以提高可读性。对于其他地图的结果, 请参见附录中的图 3。

为当前操作类型选择不正确的参数。为了简单起见, 我们没有提供功能完整的无效动作掩码, 因为 *Source Unit* 和 *Attack Target Unit* 上的掩码已经显着减少了动作空间。3. 天真的无效动作掩蔽。在每个时间步骤 t , 代理在 *Source Unit* 和 *Attack Target Unit* 上接收相同的掩码, 如无效操作掩码所述。仍应根据等式 4 中计算的重新归一化概率对动作进行采样, 这确保不会采样到无效动作, 但根据等式 2 中计算的概率更新梯度。我们将此实现称为 *naive invalid action masking*, 因为它的梯度不会用零替换无效动作对应的 logits 的梯度。4. 去除遮蔽。在每个时间步骤 t , 代理在 *Source*、*Unit* 和 *Attack Target Unit* 上接收相同的掩码, 如针对无效动作屏蔽所述, 并以与在无效动作屏蔽下训练的代理相同的方式进行训练。然而, 我们随后在不提供面罩的情况下评估该代理。换句话说, 在这种情况下, 我们评估如果我们使用面具进行训练, 然后在没有面具的情况下执行会发生什么。

我们在尺寸为 4×4 、 10×10 、 16×16 和 24×24 的地图中评估代理的性能。所有地图都为每个玩家提供一个基地和一个工作人员, 并且每个工作人员都位于资源附近。

4.4 评估指标

我们使用以下指标来衡量实验中代理的表现: r_{episode} 是过去 10 个剧集的平均剧集奖励。 a_{null} 是选择过去 10 集中无效的 *Source Unit* 的操作的平均数量。 a_{busy} 是选择的操作的平均数量。

Table 2: Results averaged over 4 random seeds. The symbol “-” means “not applicable”. Higher is better for r_{episode} and lower is better for a_{null} , a_{busy} , a_{owner} , t_{solve} , and t_{first} .

Strategies	Map size	r_{invalid}	r_{episode}	a_{null}	a_{busy}	a_{owner}	t_{solve}	t_{first}
Invalid action penalty	4×4	-1.00	0.00	0.00	0.00	0.00	-	0.53%
		-0.10	30.00	0.02	0.00	0.00	50.94%	0.52%
		-0.01	40.00	0.02	0.00	0.00	14.32%	0.51%
		0.00	30.25	2.17	0.22	2.70	36.00%	0.60%
	10×10	-1.00	0.00	0.00	0.00	0.00	-	3.43%
		-0.10	0.00	0.00	0.00	0.00	-	2.18%
		-0.01	0.50	0.00	0.00	0.00	-	1.57%
		0.00	0.25	90.10	0.00	102.95	-	3.41%
	16×16	-1.00	0.25	0.00	0.00	0.00	-	0.44%
		-0.10	0.75	0.00	0.00	0.00	-	0.44%
		-0.01	1.00	0.02	0.00	0.00	-	0.44%
		0.00	1.00	184.68	0.00	2.53	-	0.40%
	24×24	-1.00	0.00	49.72	0.00	0.02	-	1.40%
		-0.10	0.25	0.00	0.00	0.00	-	1.40%
		-0.01	0.50	0.00	0.00	0.00	-	1.92%
		0.00	0.50	197.68	0.00	0.90	-	1.83%
Invalid action masking	04x04	-	40.00	-	-	-	8.67%	0.07%
	10x10	-	40.00	-	-	-	11.13%	0.05%
	16x16	-	40.00	-	-	-	11.47%	0.08%
	24x24	-	40.00	-	-	-	18.38%	0.07%
Masking removed	04x04	-	33.53	63.57	0.00	17.57	76.42%	-
	10x10	-	25.93	128.76	0.00	7.75	94.15%	-
	16x16	-	17.32	165.12	0.00	0.52	-	-
	24x24	-	17.37	150.06	0.00	0.94	-	-
Naive invalid action masking	4×4	-	59.61	-	-	-	11.74%	0.07%
	10×10	-	40.00	-	-	-	13.97%	0.05%
	16×16	-	40.00	-	-	-	30.59%	0.10%
	24×24	-	38.50	-	-	-	49.14%	0.07%

a *Source Unit* that is already busy executing other actions over last 10 episodes. a_{owner} is the average number of actions that select a *Source Unit* that does not belong to Player 1 over last 10 episodes. t_{solve} is the percentage of total training time steps that it takes for the agents’ moving average episode reward of the last 10 episodes to exceed 40. t_{first} is the percentage of total training time step that it takes for the agent to receive the first positive reward.

4.5 Evaluation Results

We report the results in Figure 2 and in Table 2. Here we present a list of important observations:

Invalid action masking scales well. Invalid action masking is shown to scale well as the number of invalid actions increases; t_{solve} is roughly 12% and very similar across different map sizes. In addition, the t_{first} for invalid action masking is not only the lowest across all experiments (only taking about 0.05 – 0.08% of the total time steps), but also consistent against different map sizes. This would mean the agent was able to find the first reward very quickly regardless of the map sizes.

Invalid action penalty does not scale. Invalid action penalty is able to achieve good results in 4×4 maps, but it does not scale to larger maps. As the space of invalid action gets larger, sometimes it struggles to even find the very first reward. E.g. in the 10×10 map, agents trained with invalid action penalty with $r_{\text{invalid}} = -0.01$ spent a 3.43% of the entire training time just discovering the first reward, while agents trained with invalid action masking take roughly 0.06% of the time in all maps. In addition, the hyper-parameter r_{invalid} can be difficult to tune. Although having a negative r_{invalid} did encourage the agents not to execute any invalid actions (e.g. a_{null} , a_{busy} , a_{owner} are usually very close to zero for these agents), setting $r_{\text{invalid}} = -1$ seems to have an adverse effect of discouraging exploration by the agent, therefore achieving consistently the worst performance across maps.

表 2: 4 个随机种子的平均结果。符号 “-” 表示 “不适用”。 r_{episode} 越高越好, a_{null} 、 a_{owner} 、 t_{solve} 和 t_{first} 越低越好。

Strategies	Map size	r_{invalid}	r_{episode}	a_{null}	a_{busy}	a_{owner}	t_{solve}	t_{first}
Invalid action penalty	4×4	-1.00	0.00	0.00	0.00	0.00	-	0.53%
		-0.10	30.00	0.02	0.00	0.00	50.94%	0.52%
		-0.01	40.00	0.02	0.00	0.00	14.32%	0.51%
		0.00	30.25	2.17	0.22	2.70	36.00%	0.60%
	10×10	-1.00	0.00	0.00	0.00	0.00	-	3.43%
		-0.10	0.00	0.00	0.00	0.00	-	2.18%
		-0.01	0.50	0.00	0.00	0.00	-	1.57%
		0.00	0.25	90.10	0.00	102.95	-	3.41%
	16×16	-1.00	0.25	0.00	0.00	0.00	-	0.44%
		-0.10	0.75	0.00	0.00	0.00	-	0.44%
		-0.01	1.00	0.02	0.00	0.00	-	0.44%
		0.00	1.00	184.68	0.00	2.53	-	0.40%
	24×24	-1.00	0.00	49.72	0.00	0.02	-	1.40%
		-0.10	0.25	0.00	0.00	0.00	-	1.40%
		-0.01	0.50	0.00	0.00	0.00	-	1.92%
		0.00	0.50	197.68	0.00	0.90	-	1.83%
Invalid action masking	04x04	-	40.00	-	-	-	8.67%	0.07%
	10x10	-	40.00	-	-	-	11.13%	0.05%
	16x16	-	40.00	-	-	-	11.47%	0.08%
	24x24	-	40.00	-	-	-	18.38%	0.07%
Masking removed	04x04	-	33.53	63.57	0.00	17.57	76.42%	-
	10x10	-	25.93	128.76	0.00	7.75	94.15%	-
	16x16	-	17.32	165.12	0.00	0.52	-	-
	24x24	-	17.37	150.06	0.00	0.94	-	-
Naive invalid action masking	4×4	-	59.61	-	-	-	11.74%	0.07%
	10×10	-	40.00	-	-	-	13.97%	0.05%
	16×16	-	40.00	-	-	-	30.59%	0.10%
	24×24	-	38.50	-	-	-	49.14%	0.07%

在过去 10 集中已经忙于执行其他操作的 *Source Unit*。 a_{owner} 是过去 10 集中选择不属于玩家 1 的 *Source Unit* 的平均操作数。 t_{solve} 是智能体最近 10 个集的移动平均集奖励超过 40 所需的总训练时间步长的百分比。 t_{first} 是智能体获得第一个正奖励所需的总训练时间步长的百分比。

4.5 评价结果

我们在图 2 和表 2 中报告了结果。这里我们列出了重要的观察结果：

无效动作屏蔽可以很好地扩展。随着无效操作数量的增加，无效操作屏蔽可以很好地扩展； t_{solve} 大约为 12%，并且在不同的地图尺寸上非常相似。此外，无效动作屏蔽的 t_{first} 不仅是所有实验中最低的（仅占总时间步长的约 0.05 – 0.08%），而且对于不同的地图尺寸也保持一致。这意味着无论地图大小如何，代理都能够非常快速地找到第一个奖励。

无效动作惩罚不按比例缩放。无效动作惩罚能够在 4×4 地图中取得良好的效果，但无法扩展到更大的地图。随着无效行动的空间变得越来越大，有时甚至很难找到第一个奖励。例如。在 10×10 地图中，使用 $r_{\text{invalid}} = -0.01$ 进行无效动作惩罚训练的智能体花费了整个训练时间的 3.43% 来发现第一个奖励，而使用无效动作屏蔽训练的智能体在所有地图中大约花费了 0.06% 的时间。此外，超参数 r_{invalid} 可能很难调整。虽然负值 r_{invalid} 确实会鼓励智能体不要执行任何无效操作（例如， a_{null} 、 a_{busy} 、 a_{owner} 对于这些智能体来说通常非常接近于零），但设置 $r_{\text{invalid}} = -1$ 似乎会产生不利影响，阻止智能体进行探索，因此在整个地图上始终实现最差的性能。

KL Explosion of naive invalid action masking. According to Table 2, the r_{episode} of naive invalid action masking is the best across almost all maps. In the 4×4 map, the agent trained with naive invalid action masking even learns to travel to the other side of the map to harvest additional resources. However, naive invalid action masking has two main issues: 1) As shown in Figure 2c, 2d, the average Kullback–Leibler (KL) divergence [12] between the target and current policy of PPO for naive invalid action masking is significantly higher than that of any other experiments. Since the policy changes so drastically between policy updates, the performance of naive invalid action masking might suffer when dealing with more challenging tasks. 2) As shown in Table 2, the t_{solve} of naive invalid action masking is more volatile and sensitive to the map sizes. In the 24×24 map, for example, the agents trained with naive invalid action masking take 49.14% of the entire training time to converge. In comparison, agents trained with invalid action masking exhibit a consistent $t_{\text{solve}} \approx 12\%$ in all maps.

Masking removed still behaves to some extent. As shown in Figures 2a 2b, masking removed is still able to perform well to a certain degree. As the map size gets larger, its performance degrades and starts to execute more invalid actions by, most prominently, selecting an invalid *Source Unit*. Nevertheless, its performance is significantly better than that of the agents trained with invalid action penalty even though they are evaluated without the use of invalid action masking. This shows that the agents trained with invalid action masking can, to some extent, still produce useful behavior when the invalid action masking can no longer be provided.

5 Related Work

There have been other approaches of dealing with invalid actions. Dulac-Arnold, Evans, et al. [5] suggest to embed discrete action spaces into a continuous action space, use nearest-neighbor methods to locate the nearest valid actions. In the field of games with natural language, others propose to train an Action Elimination Network (AEN) [24] to reduce the action set.

The purpose of avoiding executing invalid actions arguably is to boost the exploration efficiency. Some less related work achieves this purpose by reducing the full discrete action space to a simpler action space. Kanervisto, et al. [10] describes this kind of work as “action space shaping”, which typically involves 1) action removals (e.g. Minecraft RL environment removes non useful actions such as “sneak” [9]), and 2) discretization of continuous action space (e.g. the famous CartPole-v0 environment discretize the continuous forces to be applied to the cart [2]). Although a well-shaped action space could help the agent efficiently explore and learn a useful policy, action space shaping is shown to be potentially difficult to tune and some times detrimental in helping the agent solve the desired tasks [5].

Lastly, Kanervisto, et al. [10] and Ye, et al. [23] provide ablation studies to show invalid action masking could be important to the performance of agents, but they do not study the empirical effect of invalid action masking as the space of invalid action grows, which is addressed in this paper.

6 Conclusions

In this paper, we examined the technique of invalid action masking, which is a technique commonly implemented in policy gradient algorithms to avoid executing invalid actions especially in domains where the action space is large. Our work shows that: 1) the gradient produced by invalid action masking is a valid policy gradient, 2) it works by applying a *state-dependent differentiable function* during the calculation of action probability distribution, 3) invalid action masking empirically scales well as the space of invalid action gets larger; in comparison, the common technique of giving a negative reward when an invalid action is issued fails to scale, sometimes struggling to find even the first reward in our environment, 4) the agent trained with invalid action masking was still able to produce useful behaviors with masking removed.

For future work, we hope to provide a better theoretical framework to explain the working mechanism of invalid action masking. In particular we plan to study whether the use of action masking has any consequences on the convergence guarantees of the learning algorithms, and design methods that could exploit masks during training but do not rely on them, in order to apply them to application domains where masks might be available during training, but not when deployed in the real world.

KL 幼稚无效动作掩蔽的爆炸。根据表 2，天真的无效动作掩码的 r_{episode} 在几乎所有地图上都是最好的。在 4×4 地图中，使用朴素无效动作掩蔽训练的智能体甚至学会前往地图的另一侧以获取额外的资源。然而，朴素无效动作屏蔽有两个主要问题：1) 如图 2c、2d 所示，朴素无效动作屏蔽的 PPO 目标和当前策略之间的平均 Kullback-Leibler (KL) 散度 [12] 明显高于任何其他实验。由于策略更新之间的策略变化如此之大，因此在处理更具挑战性的任务时，天真的无效操作屏蔽的性能可能会受到影响。2) 如表 2 所示，朴素无效动作掩码的 t_{solve} 更加不稳定并且对地图大小更敏感。例如，在 24×24 映射中，使用朴素无效动作掩蔽训练的智能体需要整个训练时间的 49.14% 才能收敛。相比之下，使用无效动作掩蔽训练的智能体在所有地图中表现出一致的 $t_{\text{solve}} \approx 12\%$ 。

移除的掩蔽在某种程度上仍然有效。如图 2a 和 2b 所示，去除掩蔽后仍然能够在一定程度上表现良好。随着地图尺寸变大，其性能会下降，并开始执行更多无效操作，最突出的是选择无效的 *Source Unit*。尽管如此，它的性能明显优于使用无效动作惩罚训练的智能体，即使它们是在不使用无效动作屏蔽的情况下进行评估的。这表明，当不再提供无效动作屏蔽时，使用无效动作屏蔽训练的智能体在某种程度上仍然可以产生有用的行为。

5 相关工作

还有其他处理无效操作的方法。杜拉克-阿诺德、埃文斯等人。[5] 建议将离散动作空间嵌入到连续动作空间中，使用最近邻方法来定位最近的有效动作。在自然语言游戏领域，其他人提出训练动作消除网络 (AEN) [24] 来减少动作集。

避免执行无效动作的目的可以说是提高探索效率。一些不太相关的工作通过将完整的离散动作空间减少到更简单的动作空间来实现此目的。卡内维斯托等人。[10] 将这种工作描述为“动作空间塑造”，其典型涉及 1) 动作移除（例如 Minecraft RL 环境移除无用的动作，例如“潜行” [9]），以及 2) 连续动作空间的离散化（例如著名的 CartPole-v0 环境离散化施加到购物车的连续力 [2]）。尽管形状良好的动作空间可以帮助智能体有效地探索和学习有用的策略，但动作空间的形状被证明可能难以调整，有时甚至不利于智能体解决所需的任务 [5]。

最后，Kanervisto 等人。[10] 和叶等人。[23] 提供了消融研究来表明无效动作掩蔽可能对代理的性能很重要，但他们没有研究无效动作掩蔽随着无效动作空间的增长而产生的经验效应，这在本文中得到了解决。

6 结论

在本文中，我们研究了无效动作屏蔽技术，这是策略梯度算法中常用的一种技术，用于避免执行无效动作，尤其是在动作空间较大的域中。我们的工作表明：1) 无效动作屏蔽产生的梯度是有效的策略梯度，2) 它通过在计算动作概率分布时应用 *state-dependent differentiable function* 来工作，3) 无效动作屏蔽根据经验缩放，并且无效动作的空间变大；相比之下，在发出无效动作时给予负面奖励的常见技术无法扩展，有时甚至很难在我们的环境中找到第一个奖励，4) 使用无效动作掩蔽训练的智能体仍然能够在移除掩蔽后产生有用的行为。

对于未来的工作，我们希望提供一个更好的理论框架来解释无效动作屏蔽的工作机制。特别是，我们计划研究动作掩码的使用是否会对学习算法的收敛保证产生任何影响，并设计可以在训练期间利用掩码但不依赖它们的方法，以便将它们应用于掩码在训练期间可能可用但在现实世界中部署时不可用的应用领域。

References

- [1] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub W. Pachocki, Michael Petrov, Henrique Pond’e de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning. *ArXiv*, abs/1912.06680, 2019.
- [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [3] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. Openai baselines. <https://github.com/openai/baselines>, 2017.
- [4] Thomas G Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of artificial intelligence research*, 13:227–303, 2000.
- [5] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin. Deep reinforcement learning in large discrete action spaces. *arXiv preprint arXiv:1512.07679*, 2015.
- [6] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep rl: A case study on ppo and trpo. In *International Conference on Learning Representations*, 2019.
- [7] Matthew Hausknecht and Peter Stone. Deep reinforcement learning in parameterized action space. *arXiv preprint arXiv:1511.04143*, 2015.
- [8] Shengyi Huang and Santiago Ontañón. Comparing observation and action representations for deep reinforcement learning in μ rts. 2019.
- [9] Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. The malmo platform for artificial intelligence experimentation. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI’16*, page 4246–4247. AAAI Press, 2016.
- [10] Anssi Kanervisto, Christian Scheller, and Ville Hautamäki. Action space shaping in deep reinforcement learning. *arXiv preprint arXiv:2004.00980*, 2020.
- [11] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [12] Solomon Kullback and Richard A Leibler. On information and sufficiency. *The annals of mathematical statistics*, 22(1):79–86, 1951.
- [13] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- [14] Marc’Aurelio Ranzato, Fu Jie Huang, Y-Lan Boureau, and Yann LeCun. Unsupervised learning of invariant feature hierarchies with applications to object recognition. In *2007 IEEE conference on computer vision and pattern recognition*, pages 1–8. IEEE, 2007.
- [15] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [17] Marius Stanescu, Nicolas A. Barriga, Andy Hess, and Michael Buro. Evaluating real-time strategy game states using convolutional neural networks. 09 2016.
- [18] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [19] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*, pages 1057–1063, 2000.

参考

- [1] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub W. Pachocki, Michael Petrov, Henrique Pond'e de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski 和 Susan 张。Dota 2 具有大规模深度强化学习。 *ArXiv*, abs/1912.06680, 2019. [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang 和 Wojciech Zaremba。开放式健身房。 *arXiv preprint arXiv:1606.01540*, 2016. [3] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu 和 Peter Zhokhov。Openai 基线。 <https://github.com/openai/baselines>, 2017. [4] 托马斯·G·迪特里奇。使用 maxq 值函数分解的分层强化学习。 *Journal of artificial intelligence research*, 13:227–303, 2000. [5] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris 和 Ben Coppin。大型离散动作空间中的深度强化学习。 *arXiv preprint arXiv:1512.07679*, 2015. [6] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph 和 Aleksander Madry。深度强化学习中的实施很重要：ppo 和 trpo 的案例研究。 *International Conference on Learning Representations*, 2019 年。 [7] 马修·豪斯克内西特和彼得·斯通。参数化动作空间中的深度强化学习。 *arXiv preprint arXiv:1511.04143*, 2015. [8] 黄胜义和圣地亚哥·翁塔尼翁。比较 μ rts 中深度强化学习的观察和动作表示。 2019. [9] 马修·约翰逊、卡佳·霍夫曼、蒂姆·赫顿和大卫·比格内尔。用于人工智能实验的马尔默平台。在 *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI'16*, 第 4246-4247 页。AAAI 出版社, 2016 年。 [10] Anssi Kanervisto, Christian Scheller 和 Ville Hautamäki。深度强化学习中的动作空间塑造。 *arXiv preprint arXiv:2004.00980*, 2020. [11] Diederik P Kingma 和 Jimmy Ba。Adam：一种随机优化方法。 *arXiv preprint arXiv:1412.6980*, 2014. [12] 所罗门·库尔贝克和理查德·A·莱布勒。关于信息和充分性。 *The annals of mathematical statistics*, 22(1):79–86, 1951. [13] 维诺德·奈尔和杰弗里·E·辛顿。整流线性单元改进了受限玻尔兹曼机。 *Proceedings of the 27th international conference on machine learning (ICML-10)*, 第 807-814 页, 2010 年。 [14] Marc' Aurelio Ranzato, Fu Jie Huang, Y-Lan Boureau 和 Yann LeCun。不变特征层次结构的无监督学习及其在对象识别中的应用。在 *2007 IEEE conference on computer vision and pattern recognition*, 第 1-8 页。IEEE, 2007. [15] 约翰·舒尔曼、菲利普·莫里茨、谢尔盖·莱文、迈克尔·乔丹和彼得·阿贝尔。使用广义优势估计的高维连续控制。 *arXiv preprint arXiv:1506.02438*, 2015. [16] 约翰·舒尔曼、菲利普·沃尔斯基、普拉富拉·达里瓦尔、亚历克·雷德福德和奥列格·克里莫夫。近端策略优化算法。 *arXiv preprint arXiv:1707.06347*, 2017. [17] Marius Stanescu, Nicolas A. Barriga, Andy Hess 和 Michael Buro。使用卷积神经网络评估实时策略游戏状态。 2016 年 09 月。 [18] 理查德·S·萨顿和安德鲁·G·巴托。 *Reinforcement learning: An introduction*。麻省理工学院出版社, 2018 年。 [19] Richard S Sutton, David A McAllester, Satinder P Singh 和 Yishay Mansour。具有函数逼近的强化学习的策略梯度方法。 *Advances in neural information processing systems*, 第 1057-1063 页, 2000 年。

- [20] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [21] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, et al. Starcraft ii: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017.
- [22] Zuozhi Yang and Santiago Ontañón. Learning map-independent evaluation functions for real-time strategy games. *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–7, 2018.
- [23] Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. Mastering complex control in moba games with deep reinforcement learning. *arXiv preprint arXiv:1912.09729*, 2019.
- [24] Tom Zahavy, Matan Haroush, Nadav Merlis, Daniel J Mankowitz, and Shie Mannor. Learn what not to learn: Action elimination with deep reinforcement learning. In *Advances in Neural Information Processing Systems*, pages 3562–3573, 2018.

[20] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Jun-young Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev 等。使用多智能体强化学习的星际争霸 2 大师级别。 *Nature*, 575(7782):350–354, 2019. [21] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michèle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser 等。《星际争霸 2》：强化学习的新挑战。 *arXiv preprint arXiv:1708.04782*, 2017. [22] 杨作志和圣地亚哥·翁塔尼翁。学习实时策略游戏的独立于地图的评估函数。 *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, 第1-7页, 2018年. [23] 叶德恒, 刘赵, 孙明飞, 石北, 赵培林, 吴浩, 于洪胜, 杨少杰, 吴西鹏, 郭庆伟, 等。通过深度强化学习掌握 Moba 游戏中的复杂控制。 *arXiv preprint arXiv:1912.09729*, 2019. [24] Tom Zahavy, Matan Haroush, Nadav Merlis, Daniel J Mankowitz 和 Shie Mannor。学习不该学的内容：通过深度强化学习消除动作。 *Advances in Neural Information Processing Systems*, 第 3562-3573 页, 2018 年。

Appendices

A Details on the Training Algorithm Proximal Policy Optimization

The DRL algorithm that we use to train the agent is Proximal Policy Optimization (PPO) [16], one of the state of the art algorithms available. There are two important details regarding our PPO implementation that warrants explanation, as those details are not elaborated in the original paper. The first detail concerns how to generate an action in the `MultiDiscrete` action space as defined in the OpenAI Gym environment [2] of gym-microrts [8], while the second detail is about the various code-level optimizations utilized to augment performance. As pointed out by Engstrom, Ilyas, et al. [6], such code-level optimizations could be critical to the performance of PPO.

A.1 Multi Discrete Action Generation

To perform an action a_t in μ RTS, according to Table 1, we have to select a Source Unit, Action Type, and its corresponding action parameters. So in total, there are $hw \times 6 \times 4 \times 4 \times 4 \times 4 \times 6 \times hw = 9216(hw)^2$ number of possible discrete actions (including invalid ones), which grows exponentially as we increase the map size. If we apply the PPO directly to this discrete action space, it would be computationally expensive to generate the distribution for $9216(hw)^2$ possible actions. To simplify this combinatorial action space, `openai/baselines` [3] library proposes an idea to consider this discrete action to be composed from some smaller *independent* discrete actions. Namely, a_t is composed of smaller actions

$$a_t^{\text{Source Unit}}, a_t^{\text{Action Type}}, a_t^{\text{Move Parameter}}, a_t^{\text{Harvest Parameter}}, \\ a_t^{\text{Return Parameter}}, a_t^{\text{Produce Direction Parameter}}, a_t^{\text{Produce Type Parameter}}, a_t^{\text{Attack Target Unit}}$$

And the policy gradient is updated in the following way (without considering the PPO’s clipping for simplicity)

$$\begin{aligned} \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t &= \sum_{t=0}^{T-1} \nabla_{\theta} \left(\sum_{d \in D} \log \pi_{\theta}(a_t^d|s_t) \right) G_t \\ &= \sum_{t=0}^{T-1} \nabla_{\theta} \log \left(\prod_{d \in D} \pi_{\theta}(a_t^d|s_t) \right) G_t \end{aligned}$$

$$D = \{\text{Source Unit, Action Type, Move Parameter, Harvest Parameter, Return Parameter,} \\ \text{Produce Direction Parameter, Produce Type Parameter, Attack Target Unit,}\}$$

Implementation wise, for each Action Component of range $[0, x - 1]$, the logits of the corresponding shape x is generated, which we call Action Component logits, and each a_t^d is sampled from this Action Component logits. Because of this idea, the algorithm now only has to generate $hw + 6 + 4 + 4 + 4 + 4 + 6 + hw = 2hw + 36$ number of logits, which is significantly less than $9216(hw)^2$. To the best of our knowledge, this approach of handling large multi discrete action space is only mentioned by Kanervisto et, al [10].

A.2 Code-level Optimizations

Here is a list of code-level optimizations utilized in this experiments. For each of these optimizations, we include a footnote directing the readers to the files in the `openai/baselines` [3] that implements these optimization.

1. **Normalization of Advantages⁴**: After calculating the advantages based on GAE, the advantages vector is normalized by subtracting its mean and divided by its standard deviation.
2. **Normalization of Observation⁵**: The observation is pre-processed before feeding to the PPO agent. The raw observation was normalized by subtracting its running mean and divided by its variance; then the raw observation is clipped to a range, usually $[-10, 10]$.

⁴<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L139>

⁵https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

附录

一个有关训练算法近端策略优化的详细信息 化

我们用来训练代理的 DRL 算法是近端策略优化 (PPO) [16], 它是最先进的算法之一。关于我们的 PPO 实施, 有两个重要细节需要解释, 因为原始论文中没有详细说明这些细节。第一个细节涉及如何在 gym-microrts [8] 的 OpenAI Gym 环境 [2] 中定义的多离散动作空间中生成动作, 而第二个细节涉及用于增强性能的各种代码级优化。正如 Engstrom、Ilyas 等人所指出的。[6], 这种代码级优化对于 PPO 的性能可能至关重要。

A.1 多离散动作生成

要在 μ RTS 中执行操作 a_t , 根据表 1, 我们必须选择源单元、操作类型及其相应的操作参数。因此, 总共有 $hw \times 6 \times 4 \times 4 \times 4 \times 4 \times 6 \times hw = 9216(hw)^2$ 个可能的离散动作 (包括无效动作), 随着地图大小的增加, 该数量呈指数增长。如果我们将 PPO 直接应用于这个离散动作空间, 则生成 $9216(hw)^2$ 个可能动作的分布在计算上会非常昂贵。为了简化这个组合动作空间, openai/baselines [3] 库提出了一种想法, 认为这个离散动作是由一些较小的 *independent* 离散动作组成的。即, a_t 由较小的动作组成

$$a_t^{\text{Source Unit}}, a_t^{\text{Action Type}}, a_t^{\text{Move Parameter}}, a_t^{\text{Harvest Parameter}}, \\ a_t^{\text{Return Parameter}}, a_t^{\text{Produce Direction Parameter}}, a_t^{\text{Produce Type Parameter}}, a_t^{\text{Attack Target Unit}}$$

策略梯度按以下方式更新 (为简单起见, 不考虑 PPO 的裁剪)

$$\begin{aligned} \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t &= \sum_{t=0}^{T-1} \nabla_{\theta} \left(\sum_{d \in D} \log \pi_{\theta}(a_t^d | s_t) \right) G_t \\ &= \sum_{t=0}^{T-1} \nabla_{\theta} \log \left(\prod_{d \in D} \pi_{\theta}(a_t^d | s_t) \right) G_t \end{aligned}$$

$D = \{ \text{来源单元, 动作类型, 移动参数, 收获参数, 返回参数,} \\ \text{产品 ce 方向参数, 生产类型参数, 攻击目标} \quad \text{t 单位,} \}$

在实现方面, 对于范围 $[0, x - 1]$ 的每个 Action Component, 都会生成相应形状 x 的 logits, 我们将其称为 Action Component logits, 并且每个 a_t^d 都是从该 Action Component logits 中采样的。由于这个想法, 算法现在只需生成 $hw + 6 + 4 + 4 + 4 + 4 + 6 + hw = 2hw + 36$ 个 logits, 这明显小于 $9216(hw)^2$ 。据我们所知, 这种处理大型多离散动作空间的方法仅由 Kanevristo 等人[10]提到。

A.2 代码级优化

以下是本实验中使用的代码级优化的列表。对于每一项优化, 我们都包含一个脚注, 引导读者查看实现这些优化的 openai/baselines [3] 中的文件。

1. 优势归一化⁴: 基于 GAE 计算出优势后, 通过减去其均值并除以其标准差来对优势向量进行归一化。

2. 观测值标准化⁵: 观测值在输入到 PPO 代理之前进行预处理。通过减去其运行平均值并除以其方差来对原始观察值进行归一化; 然后将原始观察值裁剪到一个范围, 通常为 $[-10, 10]$ 。

⁴<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L139>

⁵https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

3. **Rewards Scaling**⁶: Similarly, the reward is pre-processed by dividing the running variance of the discounted the returns, following by clipping it to a range, usually $[-10, 10]$.
4. **Value Function Loss Clipping**⁷: The PPO implementation of *openai/baselines* clips the value function loss in a manner that is similar to the PPO’s clipped surrogate objective:

$$V_{loss} = \max \left[(V_{\theta_t} - V_{targ})^2, (V_{\theta_{t-1}} + \text{clip}(V_{\theta_t} - V_{\theta_{t-1}}, -\varepsilon, \varepsilon))^2 \right]$$

where V_{targ} is calculated by adding $V_{\theta_{t-1}}$ and the A calculated by General Advantage Estimation[15].

5. **Adam Learning Rate Annealing**⁸: The Adam [11] optimizer’s learning rate is set to decay as the number of timesteps agent trained increase.
6. **Mini-batch updates**⁹: The PPO implementation of the *openai/baselines* also uses mini-batches to compute the gradient and update the policy instead of the whole batch data such as in *open/spinningup*. The mini-batch sampling scheme, however, still makes sure that every transition is sampled only once, and that the all the transitions sampled are actually for the network update.
7. **Global Gradient Clipping**¹⁰: For each update iteration in an epoch, the gradients of the policy and value network are clipped so that the “global ℓ_2 norm” (i.e. the norm of the concatenated gradients of all parameters) does not exceed 0.5.
8. **Orthogonal Initialization of weights**¹¹: The weights and biases of fully connected layers use with orthogonal initialization scheme with different scaling. For our experiments, however, we always use the scaling of 1 for historical reasons.

B Additional Details on the μ RTS Environment Setup

Each action in μ RTS takes some internal game time, measured in ticks, for the action to be completed. *gym-microrts* [8] sets the time of performing harvest action, return action, and move action to be 10 game ticks. Once an action is issued to a particular unit, the unit would be considered as a “busy” unit and would therefore no longer be able to execute any actions until its current action is finished. To prevent the DRL algorithms from repeatedly issuing actions to “busy” units, *gym-microrts* allows performing frame skipping of 9 frames such that from the agent’s perspective, once it executes the harvest action, return action, or move action given the current observation, those actions would be finished in the next observation. Such frame skipping is used for all of our experiments.

C Reproducibility

It is important to for the research work to be reproducible. We now present the list of hyperparameters used in Table 4 and the list of neural network architecture in Table 5. In addition, we provide the source code to reproduce our experiments at GitHub¹².

⁶https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

⁷<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L68-L75>

⁸<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L135>

⁹<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L160-L162>

¹⁰<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L107>

¹¹<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/a2c/utils.py#L58>

¹²<https://github.com/neurips2020submission/invalid-action-masking>

3. 奖励缩放⁶: 类似地, 奖励是通过除以折扣回报的运行方差来预处理的, 然后将其裁剪到一个范围, 通常是 $[-10, 10]$ 。4. 值函数损失裁剪⁷: *openai/baselines* 的 PPO 实现以类似于 PPO 裁剪代理目标的方式来裁剪值函数损失:

$$V_{loss} = \max \left[(V_{\theta_t} - V_{targ})^2, (V_{\theta_{t-1}} + \text{clip}(V_{\theta_t} - V_{\theta_{t-1}}, -\epsilon, \epsilon))^2 \right]$$

其中 V_{targ} 是通过将 $V_{\theta_{t-1}}$ 和一般优势估计[15]计算出的 A 相加计算得出的。5. Adam 学习率退火⁸: Adam [11] 优化器的学习率设置为随着代理训练的时间步数的增加而衰减。6. 小批量更新⁹: *openai/baselines*的PPO实现也使用小批量来计算梯度并更新策略, 而不是像*open/spinningup*中那样使用整个批量数据。然而, 小批量采样方案仍然确保每个转换仅采样一次, 并且采样的所有转换实际上都是用于网络更新。7. 全局梯度裁剪¹⁰: 对于一个epoch中的每次更新迭代, 策略和价值网络的梯度被裁剪, 使得“全局 ℓ_2 范数”(即所有参数的级联梯度的范数)不超过0.5。8. 权重的正交初始化¹¹: 全连接层的权重和偏差使用不同缩放的正交初始化方案。然而, 在我们的实验中, 由于历史原因, 我们始终使用 1 的缩放比例。

B 有关 μ RTS 环境设置的其他详细信息

μ RTS 中的每个动作都需要一些内部游戏时间(以刻度为单位)才能完成该动作。*gym-microrts* [8] 将执行收获动作、返回动作和移动动作的时间设置为 10 个游戏周期。一旦向特定单元发出操作, 该单元将被视为“繁忙”单元, 因此在当前操作完成之前将不再能够执行任何操作。为了防止 DRL 算法重复向“繁忙”单元发出动作, *gym-microrts* 允许执行 9 帧的跳帧, 这样从代理的角度来看, 一旦它执行给定当前观察的收获动作、返回动作或移动动作, 这些动作将在下一个观察中完成。我们所有的实验都使用这种跳帧。

C 再现性

研究工作的可重复性非常重要。现在, 我们列出了表 4 中使用的超参数列表和表 5 中的神经网络架构列表。此外, 我们还在 GitHub¹² 上提供了重现实验的源代码。

⁶https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/common/vec_env/vec_normalize.py#L4

⁷<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L68-L75>

⁸<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L135>

⁹<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/ppo2.py#L160-L162>

¹⁰<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/ppo2/model.py#L107>

¹¹<https://github.com/openai/baselines/blob/ea25b9e8b234e6ee1bca43083f8f3cf974143998/baselines/a2c/utils.py#L58>

¹²<https://github.com/neurips2020submission/invalid-action-masking>

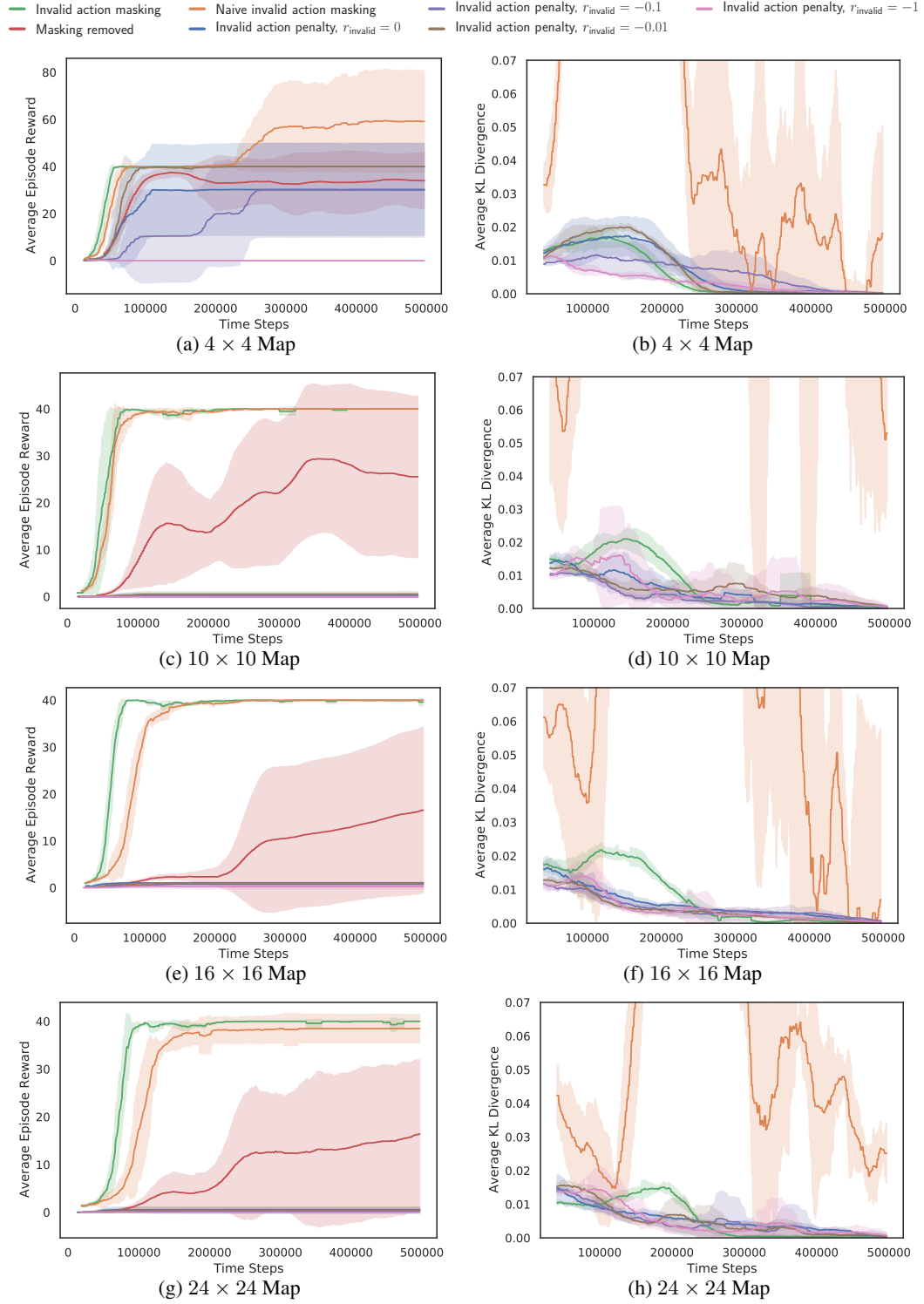


Figure 3: The left column show the learning curves of agents with different strategies to handle invalid actions. The x-axis shows the number of game steps and y-axis shows the average episode reward gathered. The right column have the x-axis showing the number of game steps and y-axis showing the average Kullback–Leibler (KL) divergence between the target and current policy of PPO. Shaded area represents one standard deviation of the data over 4 random seeds. Curves are smoothed for readability.

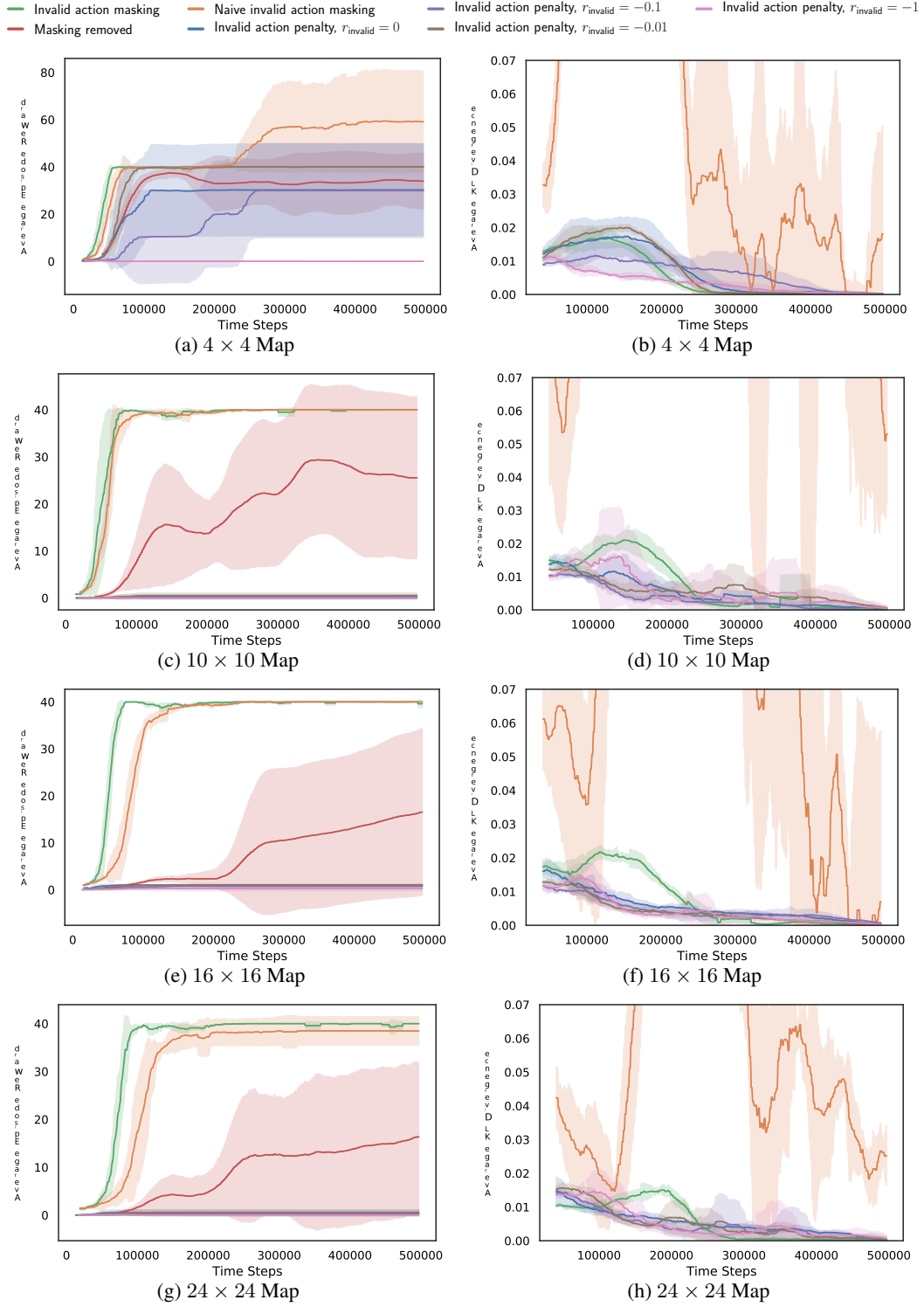


图 3: 左栏显示了采用不同策略处理无效动作的智能体的学习曲线。x 轴显示游戏步骤数, y 轴显示收集的平均剧集奖励。右列的 x 轴显示游戏步数, y 轴显示 PPO 目标和当前策略之间的平均 Kullback-Leibler (KL) 差异。阴影区域代表 4 个随机种子数据的一个标准差。曲线经过平滑处理以提高可读性。

Table 3: The list of feature maps and their descriptions.

Features	Planes	Description
Hit Points	5	0, 1, 2, 3, ≥ 4
Resources	5	0, 1, 2, 3, ≥ 4
Owner	3	player 1, -, player 2
Unit Types	8	-, resource, base, barrack, worker, light, heavy, ranged
Current Action	6	-, move, harvest, return, produce, attack

Table 4: The list of experiment parameters and their values.

Parameter Names	Parameter Values
Total Time steps	500,000 time steps
γ (Discount Factor)	0.99
λ (for GAE)	0.97
ε (PPO’s Clipping Coefficient)	0.2
η (Entropy Regularization Coefficient)	0.01
ω (Gradient Norm Threshold)	0.5
K (Number of PPO Update Iteration Per Epoch)	10
α_π Policy’s Learning Rate	0.0003
α_v Value Function’s Learning Rate	0.0003

Table 5: Neural Network Architecture. To explain the notation, let us provide detailed description of the architecture used in 24×24 map as an example. The input to the neural network is a tensor of shape $(24, 24, 27)$. The first hidden layer convolves 16 3×3 filters with stride 1 with the input tensor followed by a 2×2 max pooling layer [14] and applies a rectifier nonlinearity[13]. The second hidden layer similarly convolves 32 2×2 filters with stride 1 followed by a 2×2 max pooling layer and applies a rectifier nonlinearity. The final hidden layer is a fully connected linear layer consisting of 128 rectifier units. The output layer is a fully connected linear layer with $2hw + 36 = 1188$ number of output.

4×4	10×10
Conv2d(27, 16, kernel_size=2,), MaxPool2d(1), ReLU() Flatten() Linear(144, 128), ReLU(), Linear(128, 68)	Conv2d(27, 16, kernel_size=3,), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(1152, 128), ReLU(), Linear(128, 236)
16×16	24×24
Conv2d(27, 16, kernel_size=3), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(4608, 128), ReLU(), Linear(128, 548)	Conv2d(27, 16, kernel_size=3, stride=1), MaxPool2d(2), ReLU(), Conv2d(16, 32, kernel_size=2, stride=1), MaxPool2d(2), ReLU() Flatten() Linear(800, 128), ReLU(), Linear(128, 1188)

表 3: 特征图列表及其描述。

Features	Planes	Description
Hit Points	5	0, 1, 2, 3, ≥ 4
Resources	5	0, 1, 2, 3, ≥ 4
Owner	3	player 1, -, player 2
Unit Types	8	-, resource, base, barrack, worker, light, heavy, ranged
Current Action	6	-, move, harvest, return, produce, attack

表 4: 实验参数及其值的列表。

Parameter Names	Parameter Values
Total Time steps	500,000 time steps
γ (Discount Factor)	0.99
λ (for GAE)	0.97
ε (PPO's Clipping Coefficient)	0.2
η (Entropy Regularization Coefficient)	0.01
ω (Gradient Norm Threshold)	0.5
K (Number of PPO Update Iteration Per Epoch)	10
α_π Policy's Learning Rate	0.0003
α_v Value Function's Learning Rate	0.0003

表 5: 神经网络架构。为了解释这种表示法, 让我们以 24×24 映射中使用的架构为例进行详细描述。神经网络的输入是形状为 (24, 24, 27) 的张量。第一个隐藏层将步长为 1 的 $16 \times 3 \times 3$ 滤波器与输入张量进行卷积, 后跟 2×2 最大池化层 [14], 并应用整流非线性 [13]。第二个隐藏层同样以步长 1 卷积 $32 \times 2 \times 2$ 滤波器, 后跟 2×2 最大池化层, 并应用整流非线性。最终隐藏层是一个由 128 个整流单元组成的全连接线性层。输出层是一个全连接的线性层, 输出数量为 $2hw + 36 = 1188$ 。

4×4	10×10
Conv2d(27, 16, kernel_size=2,), MaxPool2d(1), ReLU() Flatten() Linear(144, 128), ReLU(), Linear(128, 68)	Conv2d(27, 16, kernel_size=3,), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(1152, 128), ReLU(), Linear(128, 236)
16×16	24×24
Conv2d(27, 16, kernel_size=3), MaxPool2d(1), ReLU(), Conv2d(16, 32, kernel_size=3), MaxPool2d(1), ReLU() Flatten() Linear(4608, 128), ReLU(), Linear(128, 548)	Conv2d(27, 16, kernel_size=3, stride=1), MaxPool2d(2), ReLU(), Conv2d(16, 32, kernel_size=2, stride=1), MaxPool2d(2), ReLU() Flatten() Linear(800, 128), ReLU(), Linear(128, 1188)